SQL Code

```sql
-- ================================
-- Create Database
-- ================================

DROP DATABASE IF EXISTS inventory_db;
CREATE DATABASE inventory_db;
USE inventory_db;

-- ================================
--  Supplier Table
-- ================================

CREATE TABLE Supplier (
    supplier_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    phone VARCHAR(15),
    email VARCHAR(100),
    address VARCHAR(200)
);

-- ================================
-- 📦 Product Table
-- ================================

CREATE TABLE Product (
    product_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    category VARCHAR(50),
    price DECIMAL(10,2) NOT NULL CHECK (price > 0),
    stock_quantity INT NOT NULL CHECK (stock_quantity >= 0),
    supplier_id INT,
    FOREIGN KEY (supplier_id)
        REFERENCES Supplier(supplier_id)
        ON DELETE SET NULL
        ON UPDATE CASCADE
);

-- ================================
--  Customer Table
-- ================================

CREATE TABLE Customer (
    customer_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    phone VARCHAR(15),
    email VARCHAR(100)
);

-- ================================
--  Purchase Table
```

```sql
-- ================================

CREATE TABLE Purchase (
    purchase_id INT PRIMARY KEY AUTO_INCREMENT,
    purchase_date DATE NOT NULL,
    supplier_id INT,
    FOREIGN KEY (supplier_id)
        REFERENCES Supplier(supplier_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);

-- ================================
--  Purchase Detail Table
-- ================================

CREATE TABLE Purchase_Detail (
    purchase_id INT,
    product_id INT,
    quantity INT NOT NULL CHECK (quantity > 0),
    cost_price DECIMAL(10,2) NOT NULL CHECK (cost_price > 0),
    PRIMARY KEY (purchase_id, product_id),
    FOREIGN KEY (purchase_id)
        REFERENCES Purchase(purchase_id)
        ON DELETE CASCADE,
    FOREIGN KEY (product_id)
        REFERENCES Product(product_id)
        ON DELETE CASCADE
);

-- ================================
--  Sales Table
-- ================================

CREATE TABLE Sales (
    sales_id INT PRIMARY KEY AUTO_INCREMENT,
    sales_date DATE NOT NULL,
    customer_id INT,
    FOREIGN KEY (customer_id)
        REFERENCES Customer(customer_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);

-- ================================
-- Sales Details Table
-- ================================

CREATE TABLE Sales_Details (
    sales_id INT,
    product_id INT,
    quantity INT NOT NULL CHECK (quantity > 0),
    selling_price DECIMAL(10,2) NOT NULL CHECK (selling_price > 0),
    PRIMARY KEY (sales_id, product_id),
```

```sql
    FOREIGN KEY (sales_id)
        REFERENCES Sales(sales_id)
        ON DELETE CASCADE,
    FOREIGN KEY (product_id)
        REFERENCES Product(product_id)
        ON DELETE CASCADE
);

-- ================================
-- 9 Indexes (Performance Optimization)
-- ================================

CREATE INDEX idx_product_name ON Product(name);
CREATE INDEX idx_sales_date ON Sales(sales_date);

-- ================================
--  Trigger: Auto Reduce Stock After Sale
-- ================================

DELIMITER $$

CREATE TRIGGER update_stock_after_sale
AFTER INSERT ON Sales_Details
FOR EACH ROW
BEGIN
    UPDATE Product
    SET stock_quantity = stock_quantity - NEW.quantity
    WHERE product_id = NEW.product_id;
END $$

DELIMITER ;

-- ====================
-- INSERT 5 CUSTOMERS
-- ====================

INSERT INTO Customer (name, phone, email) VALUES
('Rahul Sharma', '9000000001', 'rahul@gmail.com'),
('Amit Verma', '9000000002', 'amit@gmail.com'),
('Priya Singh', '9000000003', 'priya@gmail.com'),
('Neha Gupta', '9000000004', 'neha@gmail.com'),
('Rohit Kumar', '9000000005', 'rohit@gmail.com');

-- ====================
-- INSERT PURCHASE
-- ====================

INSERT INTO Purchase (purchase_date, supplier_id) VALUES
('2026-02-01', 1),
('2026-02-02', 2),
('2026-02-03', 3),
('2026-02-04', 4),
('2026-02-05', 5);
```

```sql
INSERT INTO Purchase_Detail (purchase_id, product_id, quantity, cost_price) VALUES
(1, 1, 5, 45000),
(2, 2, 20, 400),
(3, 3, 10, 600),
(4, 4, 5, 10000),
(5, 5, 5, 13000);

-- ====================
-- INSERT SALES
-- ====================

INSERT INTO Sales (sales_date, customer_id) VALUES
('2026-02-20', 1),
('2026-02-21', 2),
('2026-02-22', 3),
('2026-02-23', 4),
('2026-02-24', 5);

INSERT INTO Sales_Details (sales_id, product_id, quantity, selling_price) VALUES
(1, 1, 1, 52000),
(2, 2, 2, 600),
(3, 3, 1, 1000),
(4, 4, 1, 13000),
(5, 5, 1, 17000);

-- ================================
-- 1️⃣ STOCK STATUS
-- ================================

SELECT product_id, name, stock_quantity
FROM Product;

-- ================================
-- 2️⃣ COMPLETE SALES REPORT
-- ================================

SELECT
    s.sales_id,
    s.sales_date,
    c.name AS customer_name,
    p.name AS product_name,
    sd.quantity,
    sd.selling_price,
    (sd.quantity * sd.selling_price) AS total_amount
FROM Sales s
JOIN Customer c ON s.customer_id = c.customer_id
JOIN Sales_Details sd ON s.sales_id = sd.sales_id
JOIN Product p ON sd.product_id = p.product_id;

-- ================================
-- 3️⃣ TOTAL REVENUE
-- ================================
```

```sql

SELECT SUM(quantity * selling_price) AS Total_Revenue
FROM Sales_Details;

-- ================================
-- 4️⃣ TOTAL PURCHASE COST
-- ================================

SELECT SUM(quantity * cost_price) AS Total_Purchase_Cost
FROM Purchase_Detail;

-- ================================
-- 5️⃣ TOTAL PROFIT
-- ================================

SELECT
    (SELECT SUM(quantity * selling_price) FROM Sales_Details) -
    (SELECT SUM(quantity * cost_price) FROM Purchase_Detail)
AS Total_Profit;

-- ================================
-- 6️⃣ LOW STOCK ALERT
-- ================================

SELECT name, stock_quantity
FROM Product
WHERE stock_quantity < 20;

-- ================================
-- 7️⃣ TOP SELLING PRODUCT
-- ================================

SELECT
    p.name,
    SUM(sd.quantity) AS total_sold
FROM Sales_Details sd
JOIN Product p ON sd.product_id = p.product_id
GROUP BY p.name
ORDER BY total_sold DESC
LIMIT 1;

-- ================================
-- 8️⃣ MONTHLY SALES REPORT
-- ================================

SELECT
    MONTH(s.sales_date) AS month,
    SUM(sd.quantity * sd.selling_price) AS monthly_revenue
FROM Sales s
JOIN Sales_Details sd ON s.sales_id = sd.sales_id
GROUP BY MONTH(s.sales_date);
```